Project Title: Plagiarism Detector

A project Report

submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering

Submitted by

Yuvraj Singh

Roll No 25SCS1003002186

(Section: 1CSE12)

Under the supervision of

Mr. Deepanshu Moyal

Assistant Professor

IILM University, Greater Noida, Uttar Pradesh

(December 2025)

ABSTRACT

Plagiarism detection stands at the intersection of language, mathematics and computer science. This project explores how simple Natural Language Processing techniques can transform raw text into meaningful numerical structures that reveal hidden similarities across documents. Instead of relying on heavy neural architectures, the proposed system focuses on fundamental concepts that remain timeless in computational linguistics: TF IDF representation and cosine similarity.

The core contribution of the project is a lightweight, interactive web tool built using Python and Streamlit. It accepts plain text documents, processes them through a well-designed pipeline, and produces a clear similarity analysis supported by visual matrices. The system identifies copied or near-copied content, recognises paraphrasing to a reasonable degree, and provides users with an intuitive interface to understand how similarity manifests in vector space.

Beyond being a functional tool, the project acts as a gateway for students to understand how machines “read” text. The simplicity of the system makes it ideal for foundational learning, while its performance highlights the continued relevance of classical NLP methods in modern applications.

---

1 INTRODUCTION

Language is one of humanity's most expressive tools, yet also one of its hardest to analyse formally. The rise of digital learning and freely available written material has made plagiarism a common concern. When hundreds of students submit essays on similar topics, manually identifying copied passages becomes nearly impossible.

This project addresses that challenge by approaching text mathematically. If two documents share a similar distribution of meaningful words, the system can detect that relationship even without understanding context or semantics. The entire pipeline rests on three principles:

1. Text can be converted into numerical vectors

2. Document similarity can be evaluated through geometric comparison

3. A good interface makes complex analysis accessible

The result is a plagiarism detector that prioritises transparency and simplicity while still delivering effective performance.

1.1 Objectives

The project aims to:

• Build a functional plagiarism detection system using classical NLP
• Demonstrate TF IDF and cosine similarity in a practical context
• Provide an interactive platform for uploading and analysing text files
• Highlight overlapping or suspicious document pairs
• Serve as an educational model for understanding document comparison

---

1.2 Problem Statement

In academic settings, maintaining originality is essential. However, as the volume of written submissions increases, manual checking becomes unreliable. Many institutions rely on paid plagiarism detection services which may not be accessible to every student. There is a need for an open, transparent, easy-to-use system that evaluates similarities between documents quickly and accurately.

---

2. LITERATURE REVIEW

Plagiarism detection has evolved from simple text matching to sophisticated semantic analysis. Several influential approaches have shaped modern systems:

Fingerprint-based Methods

MOSS, developed by Aiken, pioneered token-based fingerprinting in code plagiarism detection. It identifies characteristic patterns within a document and compares them against others. Although powerful for structured languages, it struggles with natural language variations.

Sequence-based Approaches

Systems like JPlag use suffix trees and token sequences to detect reordered or partially modified content. These methods excel at structural comparison but do not capture deeper linguistic meaning.

Embedding-based Models

Deep learning introduced vector embeddings, where sentences or documents are represented in dense vector spaces. SBERT and similar models excel at recognising semantic similarity. However, these methods require significant computational resources, making them less practical for introductory or resource-constrained environments.

Cross-lingual Studies

Research by Potthast and others explores how plagiarism appears across languages. These approaches tackle the complexity of translation, meaning drift and multilingual vocabulary.

Among these diverse approaches, TF IDF with cosine similarity remains a staple baseline technique. Its balance of simplicity, interpretability and speed makes it ideal for small to medium scale systems and undergraduate projects.

---

3. METHODOLOGY

The system is designed around a clear and modular pipeline. Each stage transforms the input in a meaningful way, moving step by step toward a usable similarity score.

3.1 System Architecture

The architecture consists of five interconnected components:

1. **Input Acquisition**
   Users upload text documents via a Streamlit interface.

2. **Text Preprocessing**
   All documents undergo case normalisation, whitespace cleanup and character filtering.

3. **Feature Extraction**
   TF IDF converts each document into a vector based on term importance.

4. **Similarity Computation**
   Cosine similarity calculates the geometric closeness

between every document pair.

5. **Similarity Reporting**
   A matrix is generated showing how every document relates to the others.

This pipeline forms the backbone of the entire system and ensures consistent, interpretable results.

3.2 Dataset Description

The dataset consists of text files submitted manually through the user interface. Each document contains between 100 and 500 words. Some documents were deliberately written with overlapping content to evaluate the sensitivity of the system. All files are plain text to maintain consistency in formatting and processing.

3.3 Preprocessing Procedures

Preprocessing ensures that superficial formatting differences do not interfere with similarity analysis. The system performs the following transformations:

• Converts all characters to lowercase
• Removes repeated spaces, indentation irregularities and line noise
• Filters out special symbols that do not contribute to textual

meaning
• Optionally removes stop words for experimental comparison

Once cleaned, the documents are prepared for vectorization.

---

4. IMPLEMENTATION

The implementation was carried out in Python. Key components include:

TF IDF Vectorizer
Provided by scikit-learn, it extracts vocabulary from the dataset and assigns weights to each term.

Cosine Similarity Function
This function evaluates how closely aligned two document vectors are in multi-dimensional space.

Streamlit Application
The user interface was built using Streamlit, allowing real-time uploads and result display. The interface displays suspicious pairs, similarity percentages and a full similarity matrix.

The modular design ensures that the system can be extended or upgraded easily.

---

5. RESULTS AND DISCUSSION

The system was tested using several sample text files. The results showed clear differentiation across document types:

• Identical text yielded similarity near 1
• Paraphrased versions produced moderately high numbers depending on vocabulary overlap
• Completely unrelated documents scored near 0

The similarity matrix provides a complete overview of pairwise relationships, making it easy to inspect large sets of documents at once. The interactive threshold control also helps users focus on cases of concern.

Performance testing showed that the system handles multiple files quickly with negligible delay. For small academic batches, execution remains almost instantaneous.

---

6 LIMITATIONS

Although effective, the system has limitations:

• It cannot detect conceptual plagiarism where wording changes entirely
• TF IDF ignores sentence structure and meaning
• Large documents may overshadow smaller ones due to vocabulary size differences
• The system works only with text files in its current version

These limitations provide motivation for future improvements.

7 FUTURE SCOPE

There are several promising directions for expanding the system:

- Integration of sentence-level embeddings
- Expansion to PDF and DOCX support
- Cloud deployment for large dataset processing
- Cross-document highlighting of overlapping passages
- Cross-language analysis using multilingual models

Such upgrades would transform the tool from a student project into a more robust academic utility.

---

8 CONCLUSION

The plagiarism detection system developed in this project demonstrates how classical NLP techniques can uncover similarities between documents without relying on deep models. TF IDF and cosine similarity provide a simple yet powerful framework for comparing text, and the Streamlit interface allows users to interact with the system seamlessly. The project offers both a functional tool and an educational demonstration of how computational text analysis works.

REFERENCES

• Aiken, A. Research on fingerprint-based similarity detection

• Prechelt, L. Work on sequence-oriented document comparison

• Reimers, N. Embedding-based semantic similarity models

• Potthast, M. Multilingual plagiarism detection studies

• Turnitin. Articles on AI and academic integrity

• GitHub resources demonstrating TF IDF based plagiarism systems